

Based on the Advanced Editor scripts, applied steps, and data views provided in your screenshots, here is a highly professional, step-by-step breakdown of the ETL (Extract, Transform, Load) pipelines for both the **Customer Data** and **Employee Data** queries.

Query 1: Customer Data ETL Pipeline

This pipeline ingests an Excel worksheet, rectifies structural layout issues, splits multi-value string components, and derives new temporal metadata fields.

Step 1: Data Ingestion & Navigation

- **Source Extraction:** Extracted raw workbook structures from the local path using Excel.Workbook(File.Contents(...)).
- **Object Navigation:** Targeted and isolated the specific worksheet object labeled Customer Data from the workbook package.
- **Initial Schema Mapping:** Dynamically assigned initial structural types (type text) to foundational layout columns (Column2 through Column7) to prepare for row-skipping actions.

Step 2: Structural Alignment & Normalization

- **Top Row Trimming:** Stripped away the top 3 metadata rows from the dataset to properly align the actual tabular grid.
- **Header Promotion:** Promoted the top remaining data row to act as official table headers (Promoted Headers), setting PromoteAllScalars=true to guarantee seamless type evaluation.
- **Schema Enforcement:** Enforced strict data types across structural metrics, turning fields like Order ID into Int64.Type integers and setting up basic textual schemas for attributes such as Customer Name, Address, Contact Info, and Products.

Step 3: Multi-Value Delimiter Parsing

- **Contact Info Segregation:** Parsed the composite Contact Info field using a comma separator (,) to split it into two dedicated vector components: Contact Info.1 and Contact Info.2.
- **Email & Number Identification:** Assigned text types to the newly generated structures and explicitly renamed Contact Info.1 to Email and Contact Info.2 to Contact Number.

Step 4: Text Manipulation & Name Parsing

- **Component Duplication:** Cloned the original Customer Name attribute to create a safety copy (Customer Name - Copy) for future descriptive formatting.
- **String Segmentation:** Split the main Customer Name field by space characters (" ") to cleanly separate names into First Name (Customer Name.1) and Last Name (Customer Name.2).
- **Vector Clean-up:** Extracted anomalies like middle initials or trailing spaces into a temporary container (Customer Name.3) to keep the primary fields clean.

Step 5: Column Realignment & Metadata Enrichment

- **Column Reordering:** Systematically rearranged the schema into a logical hierarchy: Order ID, Full Name, First Name, Last Name, down through product descriptions and pricing metrics.
- **Temporal Feature Engineering:** Injected an analytical time-dimension by evaluating the base Date field and deriving a standalone calendar feature via Date.Year([Date]).
- **Final Grid Refinement:** Dropped the temporary residual column (Customer Name.3) to deliver a lean, production-ready dataset (Custom1).

Query 2: Employee Data ETL Pipeline

This pipeline processes an external CSV flat-file, enforces data integrity by eliminating duplicate records, handles missing/blank variables, and applies rigid sorting logic.

Step 1: Delimiter Ingestion & Layout Normalization

- **Flat-File Ingestion:** Ingested a raw comma-separated values document using Csv.Document(File.Contents(...)) configured for an explicit 8-column layout with null text quote processing.
- **Structural Evaluation:** Transformed unaligned column strings into baseline working objects and executed an initial layout-level header promotion step.
- **Blank Row Suppression:** Cleaned up structural anomalies by dropping entirely null horizontal structures via Table.RemoveColumns rules targeting residual blank footprints ({"", "_1"}).
- **Second-Pass Promotion:** Performed a secondary header promotion to perfectly align the remaining database structure.

Step 2: Attribute Normalization & Deduplication

- **Case Adjustment & Labeling:** Reassigned system-level fields like lowercase email and gender to standardized uppercase headers (Email, Gender).
- **Entity Deduplication:** Ran a deduplication pass over the Full Name field using Table.Distinct to eliminate duplicate worker records.
- **Transactional Deduplication:** Performed a second unique check on the Email column to guarantee that every row represents an entirely unique employee identity.

Step 3: Imputation & String Segmentation

- **Missing Value Imputation:** Targeted empty strings inside the Email array and replaced them with a standardized "No Email" placeholder value.
- **String Splitting:** Cloned the core employee identity row and used space-delimiter splitting rules to cleanly isolate composite text records into discrete First Name and Last Name columns.

Step 4: Metric Formatting & Order Optimization

- **Schema Harmonization:** Re-anchored column placement sequence to follow standard corporate dimensions: Name attributes first, followed by contact details, demographic info, and operational metrics (Salary, Department, Hire Date).
- **Temporal Order Optimization:** Applied an ascending sort on the Hire Date column (Order.Ascending) to arrange the dataset chronologically based on organizational tenure.
- **Financial Data Imputation:** Audited the numeric Salary column to swap out text strings like "Blank" with a numeric placeholder representation ("00"), prepping the vector for downstream math calculations.